Team Contribution Statement

Aidan

I defined the original project scenario, employee onboarding, before an industry was required. I wrote its definition, worked out the roles and participants, and drew its activity diagram. When the brief changed to require a real industry, we replaced onboarding with banking and telecom.

Implementation: I implemented the work-item state machine, `WorkItemState` and its helper functions, and fixed the state transitions as the integration tests exposed problems. I also wrote `CommandDecorator` and `RetryingCommand`, `WorkItemValidation` (including `ValidatedWorkItem`), `WorkflowLibrary`, and `CommandFactory`. For the industry packs, I wrote the factory methods for `BankingPack` and `TelecomPack`.

Testing: I wrote the tests for everything I implemented. I also wrote the test suites for the Chain of Responsibility and Template Method patterns, for `Command`, and for the `Tividy` facade.

Diagrams: I broke the engine class diagram into the 15 individual pattern-specific class diagrams in `docs/img/final/`, and committed them with the updated Visual Paradigm project. I also combined the UML diagrams into the single PDF for the report, and converted diagrams between file formats to help Jay.

Connor

My contributions to TIVIDY covered several areas including, project co-ordination and management, technical implementation of several design patterns, code reviews and quality assurance, integration and repository oversight and demo preparation.

Jay devised a phased implementation plan which I tracked group progress and co-ordinated their implementation responsibilities against. I then assisted members in understanding dependencies, blockers and outstanding tasks, communicated priorities and kept development work aligned with the deadlines set internally.

I was responsible for ensuring the correct implementation of Strategy, Facade, Command and Visitor and various methods across other shared patterns. Specifically I contributed to FlowController which is responsible for routing decisions, dependency resolution, task assignment and workflow completion evaluation. I integrated strategy into workflow execution and implemented industry pack installation and workflow startup in the Facade pattern. MacroCommand is responsible for sequential execution, undo nested commands and rollback and in concrete Commands I implemented 7 workflow commands. Lastly I committed WorkContext and ProgressVisitor which are responsible for shared services and workflow progress calculation respectively.

Reviewing and ensuring quality across PR's was also a substantial portion of my contribution ensuring that our code was assessed from multiple points of consideration, meaning I also contributed unit tests and debugging.

Finally integration, repository oversight and demonstration preparation formed part of my responsibilities which saw me audit, track dependencies , co-ordinate resolution of blockers and identifying required testing across the repository.

Ishmael

During the deciding and trial phases of the project I was responsible for Pattern mapping & justifications and Tracking of design decisions. This involves having to narrow down the best pattern for a particular

problem in the generic structure of the TIVIDY project. I had to decide for and against other patterns which seem to do or even solve the same problem. This involved analyzing an entire TIVIDY scenario and associating each problem in the system to a suitable design pattern.

My contributions during the peak development phases of the TIVIDY project involved implementing core system functionality, applying software design patterns, and developing unit and integration tests. I worked on foundational components such as the Command, Memento, Template Method, and WorkItem structures, as well as runtime functionality including action history, deadline monitoring, and the StageItem composite. I also contributed to the Decorator pattern through the implementation of WorkItemDecorator and SecureWorkItem, alongside tests for the State and Iterator patterns. Additionally, I developed tests for work items, workflow definitions, and the demo industry, and implemented 12 operations in the Tividy facade to support command execution, undo functionality, work-item queries, history and progress reporting, event signalling, time management, and access to shared services.

Jay

During foundational phases of the project, I was responsible for: core implementation; tooling; documentation and coordination of TIVIDY, including ensuring the presence of header files with Doxygen-style comments. Curated the repository automation through CI, Dockerised build tests, pull request templates and GitHub Actions to ensure the repository is predominantly self-managing, and all pull requests build and pass tests before a merge.

Implemented `engine/core` and `engine/events` packages and their unit tests and also implemented `Observers.cpp`. Provided elements of the implementation of the industry layer, including `BankingTasks.cpp`, `BankingDirector.cpp`, `BankingAdapters.cpp`, `TelecomCreditAdapter.cpp`, `TelecomNetworkAdapter.cpp`, `TelecomAssignment.cpp`, and `TelecomTasks.cpp` together with their necessary tests and a Banking pack smoke test.

Delivered the TUI implementation and an accompanying test suite. Used AI-assisted development and prompt design to produce a Python script that creates custom terminal-based logo art for client logos with fallbacks for terminals without colour support, and an 80-column limit; the art is compiled into the program, so nothing is read from disk at runtime. Handled the overall tooling and infrastructure of the project, including Dockerfile and its compatibility with GitHub Codespaces, CI workflows, the Doxygen documentation hosted on GitHub Pages, branch protection rules, `README.md` and `CONTRIBUTING.md` (which documents the team's workflow). During the final phases of the project, I consolidated and finalised the project for submission.

Managed the assembly, review, consolidation and formatting of the final report, wrote the design rationale, functional/non-functional/deferred requirements used to guide testing, implementation explanation, industry definitions and workflow documentation for both the Banking industry ("Hudson Bank") and the Telecom industry ("Warped"). Updated the pattern table to match the delivered system, delivered the final sequence and state diagrams and three of four activity diagrams. Delivered the accompanying slide deck for the live demonstration. Further collaborated with members to revise the final class diagrams to ensure that they reflected the project architecture and overall presentation quality.

Assisted in the coordination of the team throughout, tracking progress, reviewing code and pull requests, and delivering Phase 0 of the project to ensure all project foundations were adequate, thereafter providing a seven-phase implementation plan to assist with work distribution and availability for the team's questions.

Keval

My main responsibility in TIVIDY was Role E, covering Builder, Abstract Factory and Memento, together with supporting workflow-definition and task-execution functionality. I focused on the reusable engine components that allow workflows to be constructed, validated and restored across different industries. I coordinated shared work with teammates so that these components fitted their state-machine, runtime, command and industry implementations.

I implemented the workflow-definition core and structural validation. This included managing workflow metadata, ownership of the template structure and its policies, and detecting problems such as duplicate item IDs, empty stages, invalid dependencies and dependency cycles. I also checked review requirements, automatic-task constraints, deadlines and routing source IDs. Following team clarification, I kept routing-target validation within the runtime rather than extending the shared routing interface. This maintained the agreed separation between validating a reusable definition and checking decisions produced during execution.

For Builder, I implemented the standard workflow-building behaviour and contributed the shared WorkflowDirector::build() method. This allows directors to assemble nested stages, add tasks, configure policies and obtain a validated workflow definition. I paid particular attention to ownership, builder reuse and reset behaviour so that successive builds produce independent definitions. My Abstract Factory implementation contribution was the shared IndustryPack defaults, including notification, assignment and client-profile products. These defaults support the concrete industry packs developed by teammates while keeping the engine independent of industry-specific details.

My Memento contribution focused on instance-level snapshot capture and restoration. I implemented the collection of item mementos together with workflow data and instance status, allowing command undo to restore a consistent workflow state. Restoration preserves the existing runtime objects, services, observers and audit history. I also implemented the default state operations and shared task-execution behaviour, including validation, failure handling, outcome recording and reversibility bookkeeping.

During integration, I implemented an important correction for irreversible external effects. A task can now record an external commitment immediately, before later response processing might fail. This prevents an operation such as a successful banking transaction followed by a malformed response from being treated as safely reversible. The implementation records the barrier once per execution attempt and preserves it across snapshot restoration.

I supported these implementations with unit and integration tests, including eight formal Builder tests, eleven Memento tests and twenty-six Abstract Factory tests, as well as targeted irreversible-effect regressions. These cover ownership, invalid workflow structures, snapshot independence, rollback, undo boundaries and consistency between industry product families. I also addressed review feedback, reviewed teammates' pull requests and helped identify dependency and integration issues. Throughout the project, I aimed to verify behaviour using the real collaborating implementations as they became available, rather than relying solely on isolated tests.

Ruan

I was responsible for the consolidation of our class diagrams in the initial planning phase of the Tividy project. The requirement of industry was then added to the project, which caused me to revise the class diagram before our implementation began.

During the implementation phase, I was responsible for the testing of the Observer, Visitor and Command design patterns as well as the implementation of the FlowController, specifically the constructor, destructor and main functionality (along with Connor both by splitting implementation as planned and with Connor picking up some functionality in order to help me manage the work),

WorkflowInstance (along with Keval by splitting functionality into definitions, snapshots and main functionality as planned) in the general Tividy system. I was also responsible for the TelecomDirector in the telecommunication scenario for Tividy (along with Aidan by splitting the Telcom functions as planned). I was also the lead on the runtime integration for the TIVIDY implementation.

Thabo

Planning: In the planning phase I drew the work-item state diagram and the generic activity diagrams.

Implementation: I implemented `TaskItem` and `ManualTask` (the base class for work items), `DepthFirstIterator` for the Iterator pattern, and `WorkflowDefinition::createInstance`, which clones a validated definition into an independent instance. I also implemented `InstanceCommand`, the base class for the engine's undoable instance actions, and `DemoPack`, the generic demo industry pack. While working on `TaskItem` I removed memento logic to avoid overlapping with the Memento work.

Testing: I wrote tests for the Adapter pattern covering the banking and telecom connectors, the Decorator pattern, workflow-definition instances and partial definitions, `InstanceCommand`, and the demo industry. I fixed the include and incomplete-type errors that were blocking the iterator and decorator tests.